1.1 · RECAP OF MODULE 1

Module 1: three applications, six attacks

You saw each attack in theory, then as a live demo. Each attack used a different path.

RAG Application

1. Unauthorized users getting access to confidential documents.
2. Personal data of customers getting leaked via RAG

Agentic Application

3. Access to private data of CEO via an injected harmful email
4. A harmful email leading to memory poisoning attack

Multimodal Agent

5. Agent replying to harmful text - hidden inside an image.
6. Harmful instructions entering the model context via web fetch tool

Module 1 showed why AI security matters.

Module 2 gives the what : a definition, what makes llm application security so unique, and one method that every later module uses.

DEFINITION

AI Security is simply defined as

Assets

Protection of all the assets that our LLM-based application possesses from all unauthorized and malicious access.

Capabilities

Protection of all the capabilities that our LLM-based application possesses from all unauthorized and malicious access.

Assets

User data	Personal data, chat history, uploaded files.
Company documents	Knowledge base content, with per-user access rules.
Credentials + system prompt	API keys, tokens, hidden rules, business logic in the prompt.

Capability

`Memory`	Long-term notes the agent reads in later sessions.
`Tools, scopes, permissions`	What the agent is allowed to call, and with which rights.
`Resources behind tools`	Mailboxes, databases, file systems, payment systems.

THE OPEN QUESTION

Why do we need to study AI security separately?

WHAT IS NEW

Unique Challenges in AI Security for LLM based applications

One channel

Instructions and data travel in the same text. There is no parameterized query for prompts.

Not deterministic

The same input can give different outputs. A test that passes once proves little.

Natural language

An attacker can write one attack in unlimited forms. A block list cannot catch all forms.

Autonomy

An agent selects tools and acts without a human check at each step.

Now lets discuss - what these attacker led attacks are called in technical terms ?

These attacker led attacks are called Adversarial Attacks

DEFINITION

Adversarial attacks are deliberate attempts by an attacker to manipulate an AI system's inputs, data, model, memory, or connected tools so the system serves the attacker's goal instead of its intended goal.

IN SIMPLE WORDS

Someone purposely gives or plants something harmful so the AI leaks information, follows the wrong instruction, or takes an action it should not take.

NIST AI 100-2e2025: Adversarial Machine Learning · OWASP LLM01:2025 Prompt Injection

Examples of Adversarial Attacks

Direct prompt injection, jailbreak	The user types instructions that override the rules.
Indirect prompt injection	Instructions hide in documents, web pages, emails, images.
System prompt extraction	The attacker makes the model reveal its hidden rules.
Data exfiltration	Data leaves through a tool call, a link, or an image URL.
Memory poisoning	The attacker writes instructions into long-term memory.
Knowledge base poisoning	The attacker adds documents that the retriever will return.

Now here is our RAG based application which we made for our banking client

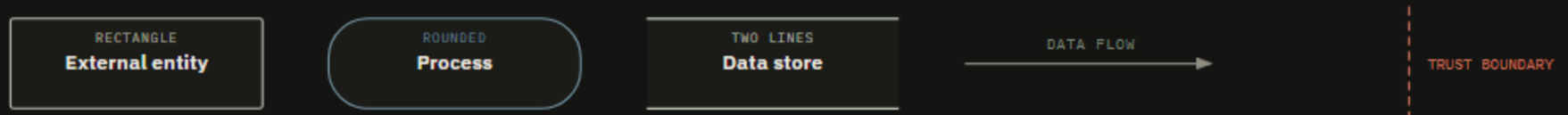
Step 1	A user asks: “Give me list of all the customers who applied for loan but got rejected.”	request
Step 2	The application searches its knowledge base without checking which records this user is allowed to access.	missing access check
Step 3	The search returns confidential customer records and passes them to the LLM as context.	unauthorized retrieval
Step 4	The LLM uses those records to answer. The user receives private customer details they should not see.	personal data leak

Now you are being asked to fix this security issue.

Tool 1: the DFD

DEFINITION LEVEL

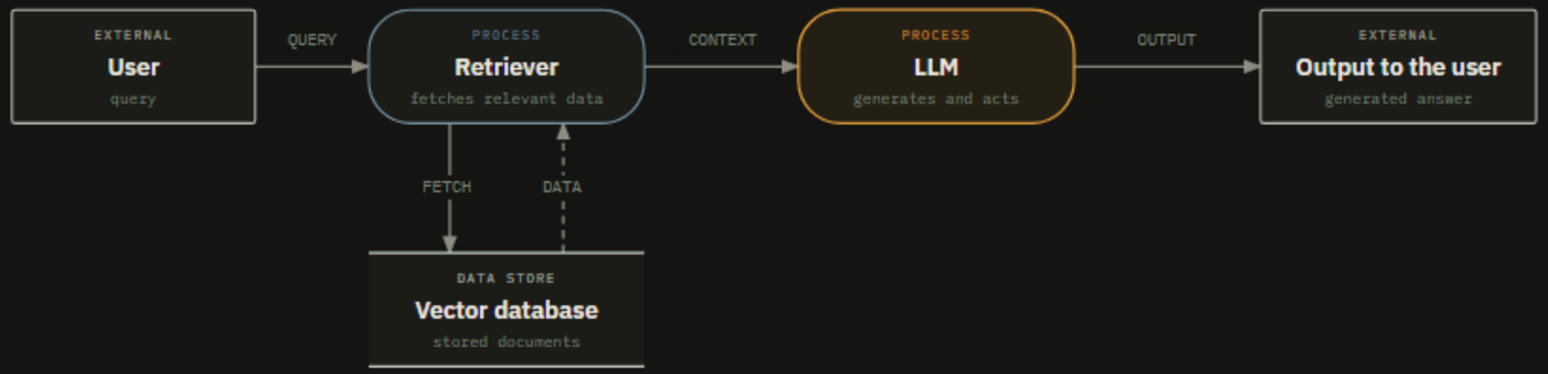
A Data Flow Diagram has five elements



user, attacker, 3rd-party API	LLM, orchestrator, retriever, tool	vector DB, memory, logs	prompt, chunks, tool call	trust level changes here

A DFD shows how data moves. It does not show code, classes, or servers.

This is how the DFD diagram for our RAG based application would look like



Tool 2: the capability lens

COMPONENTS ARE NOT EQUAL

Does each component has same capabilities in our application ?

Retriever

Reads chunks from the vector DB.
It cannot write, update, or delete.
It cannot send anything out.

vs

Email tool

Reads the inbox. Writes drafts.
Sends messages to any address.
Acts with the user's identity.

A FAIR QUESTION

Do we analyze the retriever for write risks?

No. We analyze each component only for the capabilities that it has.

QUESTIONS PER CAPABILITY

Each capability comes with one question

READ	What can it read that this user must not see?
WRITE / UPDATE	What can a wrong write change, and can we undo it?
DELETE	What is lost if it deletes the wrong item?
EXECUTE	What can the code reach: files, network, secrets?
SEND OUT	Can it send private data to an outside address?
UNTRUSTED INPUT	Can an attacker control what it receives?

Now we have the required tools to analyze any LLM based application.

Threat Modeling Framework

It is simply defined as a step by step framework via which we can identify all the weaknesses of our applications, implement the required secure measures adn then test them.

THE FULL METHOD

Five Steps for any LLM application

STEP 1	Draw the DFD of the application.	where
STEP 2	Mark entry points, exit points, and trust boundaries.	where
STEP 3	Make a capability table for each component.	what
STEP 4	Ask the question for each capability.	what
STEP 5	Write one control for each path and test it	fix